MicroGoals – Secure Goal Management Backend API

Project Overview

MicroGoals is a secure backend RESTful API developed using Node.js, Express.js, and MongoDB. It enables users to register, log in, and manage their personal goals through CRUD operations. The system uses JWT authentication and middleware to ensure data security and user-specific access control.

The system includes:

- User Registration & Login
- JWT-based Authentication
- CRUD Operations for Goals
- Protected Routes
- Centralized Error Handling

Project Instruction

- The project follows REST API principles.
- Authentication is handled using JWT.
- MongoDB is used as the database.
- APIs are tested using Thunder Client.
- The project follows MVC architecture.

Pre-requisites

Before running the project, the following tools are required:

- Node.js (v18 or higher)
- Express.js
- MongoDB (Local / Atlas)
- VS Code
- Thunder Client (Extension)
- Basic knowledge of JavaScript & REST APIs

PROJECT ARCHITECTURE

TECHNICAL ARCHITECTURE:

- Runtime: Node.js
- Framework: Express.js
- Database: MongoDB
- Authentication: JWT
- Architecture Pattern: MVC

FEATURES

- User Registration
- Secure Login with JWT
- Create Goal
- Update Goal

- Delete Goal
- View User-specific Goals
- Protected Routes
- Centralized Error Handling

USERS FLOW

1. User registers
2. User logs in
3. JWT token generated
4. User sends token in header
5. Access protected routes
6. Perform CRUD operations

PROJECT SETUP AND CONFIGURATION

Folder Structure

```
MicroGoals/
├── middleware/
│   ├── authMiddleware.js
│   └── errorMiddleware.js
├──Schema/
├──User.js
├──GoalSchema.js
```

```
├── models/
│   ├── userModel.js
│   └── goalModel.js
│
├── routes/
│   ├── authRoutes.js
│   └── goalRoutes.js
│
├── .env
├── Index.js
└── package.json
```

Project Setup

1. npm init -y

2. Install dependencies:

```
npm install express mongoose jsonwebtoken bcryptjs dotenv
```

3. Start server:

```
node server.js
```

BACKEND DEVELOPMENT

- Created Express Server

An Express server was set up to handle API requests, connect routes, and enable JSON parsing for backend operations

- Connected MongoDB

- Implemented Authentication APIs

User registration and login APIs were created with password hashing and JWT token generation for secure authentication.

- Implemented Goal CRUD APIs

Goal CRUD APIs allow authenticated users to Create, Read, Update, and Delete their personal goals. These APIs follow RESTful architecture to manage data in a structured and secure way.

- The **Create API** allows users to add new goals to the database. Each goal is linked to a specific user using a **userId**, ensuring that goals belong only to that user.
- The **Read API** allows authenticated users to retrieve only their own goals. The system filters data using the authenticated user's ID to prevent unauthorized access.
- The **Update API** allows users to modify their existing goals. Validation checks ensure that only the owner of the goal can update it.
- The **Delete API** allows users to permanently remove their goals from the system. Security checks make sure users cannot delete goals belonging to others.
- All CRUD routes are protected using **Authentication Middleware**, which verifies the JWT token before allowing access. If the token is missing or invalid, the system returns a **401 Unauthorized** error.
- An **Error Middleware** is also implemented to handle errors centrally. It captures runtime and validation errors and returns consistent JSON responses with proper HTTP status codes like **200, 201, 400, 401, and 404**.

- This implementation ensures secure data handling, proper authorization, and reliable API functionality for goal management.

```
const express = require("express");
const router = express.Router();
const goalModel = require("../Model/GoalModel");
const authmiddleware = require("../Middlewares/authMiddleware");

router.post("/create", authmiddleware, async (req, res, next) =>
{
try {
const { title } = req.body;
const goal = new goalModel({
title: title,
user: req.user,
});
await goal.save();
res.status(200).json({
success: true,
data: goal,
});
} catch (err) {
return next(err);
}
});

router.get("/:id", authmiddleware, async (req, res, next) => {
try {
const id = req.params.id;
const data = await goalModel.findOne({
_id: id,
user: req.user,
});
res.status(200).json({
success: true,
data: data,
```

```
});
} catch (error) {
return next(error);
}
});

router.put("/:id", authmiddleware, async (req, res, next) => {
try {
const goal = await goalModel.findOneAndUpdate(
{ _id: req.params.id, user: req.user },
{ completed: req.body.completed },
{ new: true },
);
res.status(200).json({
success: true,
message: "the goal is completed",
});
} catch (error) {
return next(error);
}
});

router.delete("/:id", authmiddleware, async (req, res, next) => {
try {
await goalModel.findOneAndDelete({
_id: req.params.id,
});

res.status(200).json({ message: "Goal deleted" });
} catch (error) {
next(error);
}
});

module.exports = router;
```

- Added Auth Middleware
- Auth middleware was implemented to verify JWT tokens and secure protected routes from unauthorized access.

- Added Error Middleware

An error middleware was added to handle application errors centrally and return proper status codes and messages.

DATABASE DEVELOPMENT

CONFIGURE MongoDB

- Use MongoDB Atlas or Local MongoDB

- Store connection string in `.env`

CREATE DATABASE CONNECTION

Index.js file:

- Use mongoose.connect()

```
//mongoDB connection
mongoose
  .connect(Mongo_Url)
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));
```

- Handle connection errors

CREATE SCHEMA AND MODELS

- User Schema

The User Schema defines the user data structure with name, email (unique), and hashed password fields.

```js
const mongoose = require("mongoose");
const Schema = mongoose.Schema;
const userSchema = new Schema({
  name: {
    type: String,
    required: true,
  },
  email: {
    type: String,
    unique: true,
  },
  password: {
    type: String,
    required: true,
  },
});
module.exports=userSchema;
```

- Goal Schema (with userId reference)

The Goal Schema stores goal details and includes a userId reference to associate each goal with a specific user.

PROJECT EXECUTION

Project Execution Steps

1. Start MongoDB
2. Run server
3. Test APIs in Thunder Client

4. Register User
5. Login and get JWT
6. Perform CRUD operations

Project Output Screenshots

- Successful User Registration(Signup/Login)
- In this Api call the user register or login and token was genrated
- JWT Token Response

After successful login user the api generate the token and these token was sent with every request to every api

- Goal Created Successfully
- After the login and token is generated the user give post request to api for creating there goals and goal was save in MongoDB which was the database
- Protected Route Working
- If the user does not have token which was generated by backend api then he cannot access the api so Route are protectedZ

Project Demo Video

Record video showing:

- Registration
- Login
- Token usage
- CRUD operations
- Error handling